

Laboratorio 10 · Auditoría y cifrado en reposo con Percona Server for MongoDB

Duración: 25 minutos · **Carpeta de trabajo:** `dia3/lab10-auditoria-cifrado/`

Objetivos

Vas a comprobar con tus propios ojos qué significa "cifrado en reposo" comparando los ficheros de datos de un servidor cifrado y de uno sin cifrar, y vas a configurar y leer un registro de auditoría que deja constancia de quién se autentica, quién cambia usuarios y quién consulta una colección sensible.

Contexto: ¿por qué Percona Server?

La auditoría, el cifrado en reposo y la autenticación LDAP/Kerberos **no están disponibles en MongoDB Community**; forman parte de MongoDB Enterprise, que requiere licencia comercial. **Percona Server for MongoDB (PSMDB)** es una distribución de código abierto, compatible a nivel de binario y de protocolo con MongoDB Community, que incluye estas funciones sin coste. Es la misma empresa que desarrolla las herramientas de backup (PBM) y monitorización (PMM) que veremos el día 4.

Abre `compose.yml` y localiza:

1. Cómo se genera la clave maestra y con qué permisos.
2. Los parámetros `--enableEncryption` y `--encryptionKeyFile`.
3. Los parámetros de auditoría: destino, formato, ruta y filtro.

Paso 1 · Arrancar el entorno

Libera memoria parando el replica set y arranca este laboratorio:

```
cd entorno && docker compose stop && cd ..
cd dia3/lab10-auditoria-cifrado
docker compose up -d
docker compose ps
docker compose logs psmdb | grep -iE "encrypt|audit" | head
```

Paso 2 · Datos sensibles en los dos servidores

```
docker compose exec toolbox bash
```

Inserta el mismo dato "secreto" en los dos servidores y fuerza un checkpoint para que llegue a los ficheros de datos:

```
for U in "$PS" "$CE"; do
  mongosh "$U" --quiet --eval '
    const rrhh = db.getSiblingDB("rrhh");
    rrhh.nominas.insertMany(Array.from({ length: 200 }, (_, i) => ({
      empleado: "E" + i, iban: "ES91-NOMINA-SECRETA-" + i, salario: 30000 + i
    })));
    db.adminCommand({ fsync: 1 });
    print("ok");'
done
exit
```

Paso 3 · Buscar el secreto en disco

Busca la cadena `NOMINA-SECRETA` directamente en los ficheros de WiredTiger y del journal de cada servidor. Primero en el servidor Community:

```
docker compose exec comunidad bash
grep -a -l "NOMINA-SECRETA" /data/db/*.wt /data/db/journal/* 2>/dev/null
exit
```

Y después en Percona Server con cifrado:

```
docker compose exec psmdb bash
grep -a -l "NOMINA-SECRETA" /data/db/*.wt /data/db/journal/* 2>/dev/null
exit
```

1. ¿En qué servidor aparece el texto en claro?
2. ¿Qué protege exactamente el cifrado en reposo? ¿Y qué **no** protege?

▼ Ver solución

En el servidor Community el texto aparece en algún fichero `collection-*.wt` y/o en el journal (la compresión snappy no oculta el texto, solo lo compacta). En PSMDB no aparece en ningún sitio: WiredTiger cifra cada página con AES-256 antes de escribirla, y también cifra el journal. El cifrado en reposo protege frente al robo de discos, de copias de ficheros o de snapshots de volumen. **No** protege frente a quien se conecta a la base de datos con

credenciales válidas (para eso están la autenticación y los roles), ni frente a quien lee la memoria del servidor, ni los datos en tránsito (para eso está TLS). Tampoco cifra los ficheros que MongoDB escribe fuera del motor de almacenamiento, como el log o el fichero de auditoría.

Paso 4 · La clave maestra

```
docker compose exec psmdb ls -l /enc/  
docker compose exec toolbox mongosh  
"mongodb://admin:CursoMongo2026@psmdb:27017/?authSource=admin" --quiet -  
-eval "db.serverCmdLineOpts().parsed.security"
```

Pregunta: ¿qué pasaría si perdieras el fichero `/enc/mongodb-key` ? ¿Y si alguien se lleva el volumen de datos y el volumen de la clave?

▼ Ver solución

Sin la clave maestra los datos son irrecuperables, incluidos los backups físicos de ese nodo. Y si la clave está junto a los datos, el cifrado no sirve de nada. Por eso en producción la clave no se guarda en un fichero local sino en un gestor de secretos (**HashiCorp Vault** o un servidor **KMIP**), que además permite **rotar la clave maestra** sin reescribir los datos: PSMDB ofrece `--vaultRotateMasterKey` y `--kmipRotateMasterKey`. Con una clave en fichero local, la rotación exige resincronizar cada nodo desde cero con la clave nueva, uno a uno.

Paso 5 · Auditoría

Genera actividad variada en PSMDB desde el toolbox:

```
docker compose exec toolbox bash  
mongosh "$PS" --quiet --eval '  
  db.getSiblingDB("admin").createUser({ user: "rrhh_lector", pwd:  
  "Rrhh2026", roles: [ { role: "read", db: "rrhh" } ] });  
  db.getSiblingDB("tienda").productos.insertOne({ nombre: "no auditado"  
});  
  db.getSiblingDB("tienda").productos.drop();'  
mongosh "mongodb://rrhh_lector:Rrhh2026@psmdb:27017/rrhh?  
authSource=admin" --quiet --eval 'db.nominas.find({ empleado: "E7"  
}).toArray().length'  
mongosh "mongodb://rrhh_lector:ClaveMala@psmdb:27017/rrhh?  
authSource=admin" --quiet --eval 'db.nominas.countDocuments()'  
exit
```

Lee el registro de auditoría desde dentro del servidor:

```
docker compose exec psmdb bash
tail -n 8 /data/db/auditLog.json
grep -cE '"atype" ? : ?"authenticate"' /data/db/auditLog.json
grep -E '"result" ? : ?18' /data/db/auditLog.json
exit
```

1. ¿Aparece el `insertOne` sobre `tienda.productos`? ¿Y el `drop`? ¿Por qué?
2. ¿Qué información queda registrada de la consulta de `rrhh_lector` sobre las nóminas?
3. ¿Qué significa `"result":18`?

▼ Ver solución

El `insertOne` sobre `tienda.productos` no aparece porque el filtro solo registra eventos `authCheck` (comprobaciones de autorización de operaciones CRUD) sobre `rrhh.nominas`; el `drop` sí, porque `dropCollection` está en la lista de eventos de administración. De la consulta a las nóminas queda quién (`users`), desde dónde (`remote`), cuándo (`ts`), sobre qué colección (`params`) y el comando con sus argumentos. Esto último es importante: **el fichero de auditoría puede contener datos sensibles** (filtros, documentos insertados), así que hay que protegerlo y rotarlo como cualquier otro dato confidencial. El código 18 es `AuthenticationFailed`: el intento con la contraseña incorrecta.

El filtro de auditoría es la clave de un buen diseño: auditar todo (`auditAuthorizationSuccess=true` sin filtro) genera volúmenes enormes y penaliza el rendimiento. Lo habitual es auditar siempre los eventos de administración y autenticación, y los CRUD solo sobre las colecciones sensibles.

Para ampliar: autenticación LDAP

PSMDB también permite autenticar usuarios contra un directorio LDAP o Active Directory, de dos formas: con `saslauthd` como intermediario (mecanismo `PLAIN`) o con la integración nativa, que además permite **autorización LDAP**, es decir, asignar roles de MongoDB a partir de los grupos del directorio. Un fragmento de configuración típico:

```
security:
  authorization: enabled
  ldap:
    servers: "ldap.empresa.local"
    transportSecurity: tls
    bind:
      queryUser: "cn=mongodb,ou=servicios,dc=empresa,dc=local"
      queryPassword: "*****"
    userToDNMapping: '[{ match: "(.+)", substitution: "uid={0},ou=personas,dc=empresa,dc=local" } ]'
```

```
authz:
  queryTemplate: "ou=grupos,dc=empresa,dc=local??sub?(&
(objectClass=groupOfNames)(member={USER}))"
setParameter:
  authenticationMechanisms: "PLAIN,SCRAM-SHA-256"
```

Los roles se crean en la base de datos `admin` con el nombre del grupo LDAP (por ejemplo `cn=dba,ou=grupos,dc=empresa,dc=local`) y los usuarios se autentican en `$external` con `authMechanism=PLAIN`. Siempre con TLS hacia el servidor LDAP, porque PLAIN envía la contraseña al servidor MongoDB.

Para terminar

No borres este entorno: lo usaremos en el laboratorio 11 para hacer un *hot backup*.

```
docker compose stop
cd ../../entorno && docker compose up -d
```